

■ Let's test our understanding

Mitsiu Alejandro Carreño Sarabia



Agenda

- └─ Recap
- └─ Exercises

Recap

Before I forget



■ Recap

I really want to encourage you to ask questions, so:

I am offering two free tokens:

- To the person that remind's me to setup askqueue and share the link
- To the person that remind's me to check askqueue if an hour has passed and I havent checked



■ Recap

Learning material:

1. This slides
2. https://www.youtube.com/watch?v=WgJ2FUW1miA&list=PLuGpJqnV9DXq_ItwUoJ0Gk_uCr72Yvzb
3. <https://elmprogramming.com/>
4. (Official guide) <https://guide.elm-lang.org/>
5. (Official docs) <https://package.elm-lang.org/packages/elm/core/latest/>
6. (Advanced level - Standard ML)
<https://www.youtube.com/watch?v=jjX68oHAW-Y&list=PLsydD1kw8jng2t2G8USQNLz0faYZetPnH>

■ Recap

Our next tests:

- 2nd Partial = 30%
 - Class exercise / Homework = 10%
 - Theoretical evaluation = 40%
 - Practical evaluation (paper based, NO computer!) = 50%

■ Recap (Unit 1)

- Command to print current working directory
 - `pwd`
- Command to change current terminal to the parent directory
 - `cd ..`
- Command to change to the following path: `C:\Users\Yo\Web-elm`
 - `cd C:\Users\Yo\Web-elm`
- Command to create a new elm project
 - `elm init`
- What is an expression?
 - A set of one or many constants, variables, operators or functions

■ Recap (Unit 1)

- What is a value?
 - A final answer, cannot be reduced any more
- What is computer state?
 - The value of all variables in a program at a given time
- How does elm manage state during execution?
 - State is immutable, all variables remain binded to a single value
- Which primitives exists in elm?
 - Float, Bool, Char, String, List a, Int, Tuples

■ Recap (Unit 1)

- Which data type structure must have the following expression:

```
e1 ++ e2
```

$e1 ++ e2 : \text{appendable}$

if

$e1 : \text{appendable}$

and

$e2 : \text{appendable}$

■ Recap (Unit 1)

- Which data type structure must have the following expression:

```
case e1 of  
  e2 -> e3
```

$$\begin{array}{l} \text{case } e1 : \alpha \text{ of} \\ \quad pat1 : \alpha \rightarrow \\ \qquad e2 : \beta \\ \quad pat2 : \alpha \rightarrow \\ \qquad e3 : \beta \end{array}$$

■ Recap (Unit 1)

- Which data type structure must have the following expression:

```
if e1 then
  e2
else
  e3
```

```
if e1 : Bool then
  e2 :  $\alpha$ 
else
  e3 :  $\alpha$ 
```

■ Recap (Unit 1)

- Which data type structure must have the following expression:

```
e1 + e2
```

$e1 + e2 : \textit{number}$

if

$e1 : \textit{number}$

and

$e2 : \textit{number}$

■ Recap (Unit 1)

- Which data type structure must have the following expression:

```
e1 :: e2
```

$x :: xs : \text{List } \alpha$
if
 $x : \alpha$
and
 $xs : \text{List } \alpha$

■ Recap (Unit 1)

- In elm what does semicolon (;) means?
 - Has type
- Which command allows us to implement (modifying our source code) elm format rules?
 - `elm-format src/`
- Which command allows us to verify if our code implements elm format rules?
 - `elm-format src/ --validate`
- Which command allows us to verify if the content of any .elm file complies with elm syntax rules?
 - `elm-make src/*`
- Command to start an interactive elm session
 - `elm repl`

■ Recap (Unit 1)

- What's the difference between a function definition and a function application (invoke)?
 - A function definition explains the transformation the function will do, but the actual transformation with concrete values happens until the function is applied.
- Which is another way to write the following expression:

```
'a' :: 'b' :: 'c' :: []
```

- ['a', 'b', 'c']
- In elm a list can be either one of two things which are they?
 - Nil [] (an empty list)
 - Cons :: (something attached to a list)

■ Recap (Unit 1)

- What does this type annotation means:

```
List.map : (a -> b) -> List a -> List b
```

- List.map is a function with 2 parameters (inputs), the first parameter is a function that can transform some data type (alpha) to some other representation (beta), the second parameter is a list of alphas, List.map takes each single element in the input list, pass it to the function and store the output in a new list.
- What does High-order functions means?
 - It means that functions receive the same treatment as variables, and can be passed to other functions or returned from other functions just like any other parameter.

■ Recap (Unit 1)

What does the following code means line by line?

```
multip : Int -> Int -> Int  
multip x y = x * y
```

- The first line is the type annotation, it says that `multip` is a function (has arrows) that receives two `Int` values and return a `Int` value. The second line name the two variables as "`x`" and "`y`" and defines `multip` as the operation "`x * y`"

■ Recap (Unit 1)

What does the following code means line by line?

```
foo : Bool -> List Float -> List Float -> List Float
foo a b c =
  if a then
    3.5 :: b
  else
    c
```

- The first line is the type annotation, it says that foo is a function (has arrows) that receives 3 parameters, the first is of type Bool, the second is of type List Float and the third is of type List Float, finally the foo function returns a List of Floats. The foo function names the three variables as "a", "b" and "c" it evaluates "a" of type Bool if it's true the value 3.5 is added at the front of the list b and returned, if "a" is false the list c is returned.

■ Recap (Unit 1)

```
flipper x =  
    not x      - - Invierte un boolean True => False && False =>  
    True  
  
flipAll funcTrans list =  
    List.map funcTrans list
```

1. What's the type annotation of flipper?
 - flipper : Bool -> Bool
2. What's the type annotation of flipAll?
 - flipAll : (Bool -> Bool) -> List Bool
3. How do we apply flipAll to get the result [True, False, True]
 - flipAll flipper [False, True, False]

■ Recap (Unit 1) - Html

- What's the difference between html elements and attributes?
 - Html elements are visual and compose the screen, they exist for the user to see and interact, html attributes are metadata to the elements, allowing to group, modify the behaviour, or specify interaction between the user and the elements.
- Which html tags have opening and closing tags?
 - `<p></p>` | `<h1></h1>` | `<a></a>` | `<ul></ul>` | `<li></li>` | `<div></div>`
- Which html tags **don't** have opening and closing tags (void elements)?
 - `<br>` | `<img>`
- In an Html element where do we place the content?
 - Between the opening and closing tags `<p>content</p>`

■ Recap (Unit 1) - Html

- In an Html element where do we place the attributes
 - In the opening tag <p class="value">content</p>
- All html elements have the following type annotation, what does it means?

```
<function> : List (Html.Attribute msg) -> List (Html.Html msg) ->  
            Html.Html msg
```

- It says that the function (h1, p, div, a, ...) expect's two parameters, a list of html attributes (class, id, href, ...) and a list of child html elements (h1, p, div, a, ...)

■ Recap (Unit 1) - Html

- Given the previous type annotation, which data type has main if I declare:

```
<function> : List (Html.Attribute msg) -> List (Html.Html msg) ->  
Html.Html msg
```

```
main = Html.h1 [] []
```

- `main : Html.Html msg`

■ Recap (Unit 1) - Html

In elm how do I produce the following html:

```
<p></p>
```

```
Html.p [] []
```

In elm how do I produce the following html:

```
<p class="value"></p>
```

```
Html.p [ Html.Attributes.class "value" ] []
```

■ Recap (Unit 1) - Html

In elm how do I produce the following html:

```
<p>content</p>
```

```
Html.p [] [ Html.text "content"]
```

In elm how do I produce the following html:

```
<p id="main">Content</p>
```

```
Html.p [Html.Attributes.id "main"] [Html.text "Content"]
```


■ Recap (Unit 1) - Html

In elm how do I produce the following html:

```
<p><strong></strong></p>
```

```
Html.p [] [Html.strong [][]]
```

In elm how do I produce the following html:

```
<p><strong>content</strong></p>
```

```
Html.p [] [Html.strong [][Html.text "content"]]
```

■ Recap (Unit 1) - Html

In elm how do I produce the following html:

```
<p>content<strong></strong></p>
```

```
Html.p [] [Html.text "content", Html.strong [][]]
```

In elm how do I produce the following html:

```
<p>content<strong>CONTENT</strong></p>
```

```
Html.p [] [Html.text "content", Html.strong [] [Html.text "CONTENT"]]
```

■ Recap (Unit 1) - Html

In elm how do I produce the following html:

```
<p id="main">content<strong class="bold">SubContent</strong></p>
```

```
Html.p [ Html.Attributes.id "main"]
  [
    Html.text "content"
    , Html.strong [ Html.Attributes.class "bold"]
                  [
                    Html.text "SubContext"
                  ]
  ]
```

■ Recap (Unit 2)

- How are records usefull? What possibility they offer that List or other primitives dont?
 - They allow us to group several fields to represent complex data (e.g. A person is more than just a name : String or an age: Int, or a role: UType, is the combination of all)
- Which simbol/character do we use to mark the begining and the end of a record?
 - {} Curly braces

■ Recap (Unit 2)

- What's the meaning of the following expression?

```
mit : { age:Int }  
mit = { age = 32 }  
mit.age
```

- From the variable "mit" it returns the value of the property "age"
- What's the meaning of the following expression?

```
mit : { age:Int }  
mit = { age = 32 }  
.age mit
```

- There's a function ".age" that expect's a parameter of type record with at least the property age (we send mit as that parameter)

■ Recap (Unit 2)

- What's the type annotation of the function `.age`?

```
{ b | age : a } -> a
```

- What does the following expression do:

```
{ mit | email = "mitsiu.carreno@upa" }
```

- Creates a **new record** based on all the properties in `mit`, except the `email`, which is bound to the value `"mitsiu.carreno@upa"`.
- As an important side note, `mit` remains unchanged (it's values aren't modified)

■ Recap (Unit 2)

- What does the following expression reduces to?

```
List.map .age [mit, mit, mit, mit, mit]
```

- List.map will take each element (mit : { age : Int }) and apply .age to mit, remember that .age : { b | age : a } -> a which means that .age expects a record that has a property age of an unknown type, for mit : { age : Int } we can replace alpha to { mit | age : Int } -> Int, the result will be a List [32, 32, 32]
- Which keywords does a alias use?
 - type alias
- Exemplify a type alias of Int to Entero

```
type alias Entero = Int
```

■ Recap (Unit 2)

- Exemplify a type alias of { color: String, weight: Float, type: String } to Apple

```
type alias Apple =  
  { color : String  
    , weight : Float  
    , type : String  
  }
```

Notice type alias only affect the type is not involved on values

- Which benefits does creating aliases has?
 - Using records is easier because we refer to entities like Apple, User, Computer rather than describing the whole implementation which might risk interpretation.
 - Updating records became instant and we can focus on updating values rather than types.

■ Recap (Unit 2)

- What is a component?
 - A combination of functions and `Html`, which allows us to reuse our `html` in versatile ways.
- Why repetition of code should be avoided?
 - Code repetition makes maintainability harder, as there's more code that can be flawed. We should aim to write less code to reduce the surface of potential bugs.
- How do we turn a variable into a function?
 - By adding an input and maintaining the previous type as output, also adding variables to catch the parameter values

```
anItem : Html.Html msg      anItem : String -> Html.Html msg
anItem = Html.li []         => anItem str = Html.li []
    [ Html.text "Static" ]    [ Html.text str ]
```

■ Recap (Unit 2)

- Which are the keywords used to create a new custom data type?

```
type _____  
  = Invariant 1  
  | Invariant 2  
  | Invariant 3
```

- From the previous notation, what are the invariants?
 - Are possible values of the type defined
- How can we define a data type?
 - It's a collection of possible values (possible states) (e.g. the type TrafficLight can have the values Green, Red, Yellow)

■ Recap (Unit 2)

- What are some benefits from using custom types instead of primitives?
 - Custom types allow us to fit the type and values to specific problems, primitive data types might be too constrained or flexible to adequately represent one problem which leads to interpretation.
- What is a lambda expression?
 - It's an alternative way to express a function
- What is the cost of lambda expressions?
 - As it's anonymous, it can only be referenced in the place it's created, preventing any kind of reuse.

■ Recap (Unit 2)

- What's the syntax for a lambda expression?

```
(\  _____ -> _____)
 ^ ^   ^           ^   ^
 ||   |           |   |
 || Params      Body |
 ||               |
 ||           Lambda end
Lambda start
```

Challenges

- Add to the back of a list
- Alternate strongs in li
- integrate myLaptop exer to the component aList
- List.map of a list of {url:String, content: String}